Setup for logging is done in settings.py using a configuration dictionary. At a minimum, you want a handler and a logger. Most often, you'll set up a handler for the console, a separate handler for writing to a file, and a couple of formatters. This allows you to log to different places in different hosting environments. The following listing contains a logging configuration that has both a console and file-based handler and a formatter for each, and it sends all messages in the system to both handlers.

Listing B.2 Logging configuration for both the console and a file

```
LOGGING = {
    'version': 1,
    'disable_existing_loggers': False,
    'formatters': {
        'console': {
            'format': '%(levelname)-8s %(message)s'
        },
        'file': {
            'format': \
            '%(asctime)s %(levelname)-8s %(message)s'
        }
    },
    'handlers': {
        'console': {
            'class': 'logging.StreamHandler',
            'formatter': 'console'
        },
        'file': {
            'level': 'DEBUG',
            'class': 'logging.FileHandler',
            'formatter': 'file',
            'filename': '/tmp/debug.log'
        }
    },
    'loggers': {
        '': {
            'level': 'DEBUG',
            'handlers': ['console', 'file']
        }
    }
}
```

Specifies the version of the log configuration format should always be 1

Configuration section for log formatters

A formatter named console that outputs the log level and the message

A formatter named file that outputs the current date and time, log level, and message

Configuration section for handlers

Defines a handler named console that outputs to a stream (which, by default, goes to standard-out)

The name of the formatter to use with this handler (the handler and associated formatter are usually named the same thing, but this isn't required)

Defines a handler named file that outputs to a file

Name of the formatter to use with this handler

Extra configuration used by this handler specifying the full pathname of the logging file

The empty string specifies to record messages from all loggers. This includes your own loggers as well as all those used by Django.

A log level of DEBUG catches all messages.

This logger uses both the console and file handlers. All messages will be sent to both the console and the log file.

The configuration in listing B.2 is a nice example, but it is probably not the best choice for your production system. In production, you don't want to log to console, and in development, you typically only log to console. This configuration is also trapping all log levels, which is likely a little verbose for a production setup.

The logging classes specified in the handler configurations point to objects in the Python standard library. I recommend using `logging.RotatingFileHandler` instead of `logging.FileHandler`, as it is smart enough to change to a new file when your